Ghulam Ishaq Khan Institute (GIKI)

Assignment# 2

Subject: Object Oriented Programming	**Course Code:** CS - 112 – Spring - 2025
Class: BS CY **Batch:** Fall – 2024	**Submission Date:** 17th March, 2025
Course Instructor: Muhammad Ahmad Nawaz	**Total Marks:** 10
Course TA:	
Name:	Registration Number:

Note (Read notes & instructions first)

- First, read the instructions and statements of each question carefully.
- Attempt all questions
- Write separate C++ programs for each problem.
- Include detailed comments explaining the logic and technical aspects of your code.
- Test your programs with various inputs to ensure correctness.
- Submit all your code files along
- Upload file with Registraion#_Name
-
  - **CHEATING/ CASE** will be graded as **STRAIGHT ZERO MARKS**.

Problem 1: Inheritance and Static Registry

Task:
Design a Component hierarchy for a game engine with the following requirements:

Base Class (Component):

- **Private:**
  - string id (unique identifier for the component).
- **Protected Static:**
  - static map<string, Component*> componentRegistry (tracks all components by their id).
- **Public:**
  - **Constructor:** Initializes id and adds this to componentRegistry.
  - **Static Function:** static Component* getComponent(const string& id) to retrieve components by id.
  - **Virtual Destructor:** Ensures proper cleanup of derived class objects.
  - **Pure Virtual Function:** virtual void update() = 0 (to be implemented by derived classes).

Derived Classes:

1. **PhysicsComponent:**
   - **Private:**
     - double mass (mass of the physics object).
   - **Override update():** Simulates physics behavior.
2. **RenderComponent:**
   - **Private:**
     - string texturePath (path to the texture used for rendering).
   - **Override update():** Renders graphics.

Questions:

1. Demonstrate creating PhysicsComponent and RenderComponent objects.
2. Use getComponent("physics_01") to retrieve an object and call update().
3. Explain why componentRegistry must be static.

Problem 2: Polymorphic Factory with Static Methods

Task

Implement a ShapeFactory to create polymorphic shapes using a static factory method.

Implementation

1. **Abstract Base Class (Shape):**
   - Define a pure virtual function virtual double area() = 0.
   - Add a static factory method static Shape* create(const string& type) to create Circle or Rectangle objects.
2. **Derived Classes:**
   - **Circle:**
     - Private member: double radius.
     - Override area() to calculate the area of a circle.
   - **Rectangle:**
     - Private members: double width, double height.
     - Override area() to calculate the area of a rectangle.

Questions

1. Demonstrate how to use Shape::create("Circle") to instantiate a Circle object and call its area() method.
2. Why is it necessary for the factory method create() to be static?

Problem 3: Virtual Functions and Dynamic Casting

Task:
Extend the Vehicle hierarchy to include dynamic type checking:

Base Class (Vehicle):

- **Virtual Function:** virtual string getType() { return "Vehicle"; }.

Derived Classes:

1. **Car:**
   - **Override getType():** Returns "Car".
2. **Bike:**
   - **Override getType():** Returns "Bike".

Global Function:

```
void checkType(Vehicle* v) {
  if (dynamic_cast<Car*>(v)) cout << "It’s a Car!";
  else if (dynamic_cast<Bike*>(v)) cout << "It’s a Bike!";
}
```

Questions:

1. Why is dynamic_cast necessary here?
2. What happens if getType() is non-virtual?

Problem 4: Virtual Destructors and Resource Management

Task:
Design a ResourceHandle class to manage GPU resources:

Base Class (ResourceHandle):

- **Private:**
  - int* gpuHandle (simulates a GPU resource handle).
- **Public:**

- **Constructor:** Allocates gpuHandle (simulated with new int).
- **Virtual Destructor:** Frees gpuHandle.

Derived Class (TextureHandle):

- **Private:**
  - int* textureData (simulates texture data).
- **Destructor:** Frees textureData.

Scenario:

```
ResourceHandle* tex = new TextureHandle();
delete tex; // Should free both gpuHandle and textureData.
```

Question:

1. What happens if the destructor is non-virtual? Fix the code to avoid memory leaks.

Problem 5: Polymorphic Factory with Automatic Registration

Task:
Design a Plugin system where derived classes automatically register with a factory to enable dynamic instantiation.

Base Class (Plugin):

- **Abstract with Pure Virtual Function:** virtual void execute() = 0.
- **Static Members:**
  - using Factory = std::function<Plugin*()>.
  - static std::map<std::string, Factory>& getRegistry().
  - static Plugin* create(const std::string& type).
- **Protected Constructor:** Prevents direct instantiation of Plugin.

Derived Classes (e.g., EchoPlugin):

- **Implement execute():** Defines the plugin's behavior.
- **Automatic Registration:** Uses a static Registrar to register the plugin with the factory.

Example Code Skeleton:

```
class EchoPlugin : public Plugin {
public:
  void execute() override { /* Implementation */ }

  class Registrar {
  public:
    Registrar() {
      Plugin::getRegistry()["EchoPlugin"] = []() { return new EchoPlugin(); };
    }
  };
  static Registrar registrar;
};
EchoPlugin::Registrar EchoPlugin::registrar; // Static initialization
```

Questions:

1. Explain how the Registrar ensures automatic registration without manual setup.
2. Why is the Plugin constructor protected?
3. Use Plugin::create("EchoPlugin") to instantiate and call execute().

Submission Guidelines

1. Submit all code files (.cpp, .h).
2. Include a main() function testing all edge cases.
3. Use const correctness, smart pointers (optional), and RAII principles.

4. Write a report explaining design choices (e.g., static registration mechanism).

Evaluation Criteria

1. Correct use of static members and polymorphism.
2. Understanding of factory patterns and automatic registration.
3. Memory safety (no leaks, proper destructor behavior).
4. Code structure and adherence to best practices.